Camera_Calibration

1. Coordinate systems:
    1. World coordinate system (3D)
        1. A reference (canonical) coordinate system
        2. Fixed coordinate system
        3. you can define your own one
    2. Camera coordinate system (3D)
        1. Defined for each camera
        2. Camera-relative coordinate system
    3. Image coordinate system (2D)
        1. Defined for each camera
        2. Projected space from the camera coordinate system
2. Projection models
    1. Perspective projection, more realistic (harder to get camera parameters, need accurate focal lengths (focal lengths is about zooming) and principal points (about the resolution e.g. 4k FHD) to do perspective projection)
    2. weak perspective/Orthographic (easier)
3. Calibration is the process to determine the intrinsic (K plus lens distortion) and extrinsic (R, T) parameters of a camera. For now, we will neglect the lens distortion and see later how it can be determined.
4. The solution for K, R, T can be found be applying the perspective projection equation:
    1. $\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R|T \end{bmatrix} \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$
5. There are two popular methods:
    1. Tsai's method: uses 3D objects
    2. Zhang's method: uses planar grids
6. Tsai's method: calibration from 3D objects
    1. This method was proposed in 1987 by Tsai and consists of measuring the 3D position of $n \geq 6$ control points on a 3D calibration target and the 2D coordinates of their projection in the image.
    2. Assumption: we know 2D and 3D coordinates of control points
        1. 2D: image pre-processing (e.g. corner detection).
        2. 3D: we know actual size of the 3D object and actual positions of control points as well.
    3. Applying the Direct Linear Transform (DLT)
    4. The idea of the DLT is to rewrite the perspective projection equation as a **homogeneous linear equation** and solve it by standard methods. Let's write the perspective equation for a generic 3D-2D point correspondence:
    5. $\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_u & K_{12} & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$
    6. where α_u, α_v are focal lengths, u_0, v_0 are principal points (image center), K_{12} is skew parameter (in practice skew parameter, K_{12}, must be close to 0).
    7. $\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$
    8. $\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = M \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$
    9. $\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} m_1^T \\ m_2^T \\ m_3^T \end{bmatrix} \begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix}$, where m_i^T is the i-th row of M.

10. let $\begin{bmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{bmatrix} = P$, then conversion back from homogeneous coordinates to pixel coordinates leads to:

    $u = \frac{\lambda u}{\lambda} = \frac{m_1^T \cdot P}{m_3^T \cdot P}, v = \frac{\lambda u}{\lambda} = \frac{m_2^T \cdot P}{m_3^T \cdot P} \Rightarrow$
11. $(m_1^T - u_i m_3^T) \cdot P_i = 0, (m_2^T - v_i m_3^T) \cdot P = 0$
12. further rearranging, we obtain
13. $\begin{pmatrix} P_1^T & 0^T & -u_1 P_1^T \\ 0^T & P_1^T & -v_1 P_1^T \end{pmatrix} \begin{pmatrix} \vec{m}_1 \\ \vec{m}_2 \\ \vec{m}_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
14. For n points, we can stack all these equations into a big matrix:
15. $\begin{pmatrix} P_1^T & 0^T & -u_1 P_1^T \\ 0^T & P_1^T & -v_1 P_1^T \\ & \vdots & \\ P_n^T & 0^T & -u_n P_n^T \\ 0^T & P_n^T & -v_n P_n^T \end{pmatrix} \begin{pmatrix} \vec{m}_1 \\ \vec{m}_2 \\ \vec{m}_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}$
16. $QM = 0$, Q (this matrix is known), M is the unknown matrix.
17. Minimal solution
    1. $Q_{2n \times 12}$ should have rank 11 to have a unique (up to scale) non-zero solution M.
    2. Because each 3D-to-2D point correspondence provides 2 independent equations, then $5 + \frac{1}{2}$ point correspondences are needed (in practice **6 point** correspondences)
18. Over-determined solution
    1. In theory we only need at least 5.5 point correspondences (each correspondences gives us 2 independent equations x and y) however in practice there is always noise, so we need to use overdetermined solution.
    2. For $n \geq 6$ points, a solution is the **Least squares solution**, which minimizes the sum of squared residuals, $||QM||^2$, subject to the constraint $||M||^2 = 1$. It can be solved through SVD.
    3. The solution is the eigenvector corresponding to the smallest eigenvalue of the matrix $Q^T Q$ (because it is the unit vector x that minimizes $||Qx||^2 = x^T Q^T Q x$).
    4. Matlab instructions:
    5. ```
       [U, S, V] = SVD(Q);
       M = V(:, 12);
       ```
    6. QR decomposition decomposes a matrix into orthogonal and upper triangular matrix.
    7. Once we have determined M, we can recover the intrinsic and extrinsic parameters by remembering that:
    8. $M = K(R|T)$
    9. $\begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} = \begin{bmatrix} \alpha r_{11} + u_0 r_{31} & \alpha r_{12} + u_0 r_{32} & \alpha r_{13} + u_0 r_{33} & \alpha t_1 + u_0 t_3 \\ \alpha r_{21} + v_0 r_{31} & \alpha r_{22} + v_0 r_{32} & \alpha r_{23} + v_0 r_{33} & \alpha t_2 + v_0 t_3 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix}$
    10. However notice that we are not enforcing the constraint that R is orthogonal, i.e. $RR^T = I$.
    11. To do this, we can use the so-called QR factorization of M, which decomposes M into R (orthogonal), T, and an upper triangular matrix (i.e. K).
    12. Recommendation: use many **more than 6 points** (ideally more than 20) and **non coplanar**.
    13. Another method: minimize algebraic error between the projected point and two lines intersecting at the measured point. $[\vec{x}]^\perp = \begin{bmatrix} \vec{l}_1^T \\ \vec{l}_2^T \end{bmatrix}$, $\vec{x} = P\vec{X}$, $[\vec{x}]^\perp P\vec{X} = 0$, projected point $P\vec{X}$.
    14. Let X be of size $(d+1, n)$, where X is the world points in homogeneous coordinates.
    15. solve P using:
    16. $\left(\begin{bmatrix} l_1^T \\ l_2^T \end{bmatrix} \otimes X^T \right) P = 0$, where $\otimes$ is the Kronecker product, P is 12×1, $\begin{bmatrix} l_1^T \\ l_2^T \end{bmatrix}$ is 2×3, X is 4×1, and the resulting product will result in a matrix of size 2×12. That means we need at least $2n$, $n = 6$ point correspondences to solve for camera projection matrix P.
19. Reprojection error:
    1. The reprojection error is the Euclidean distance (in pixels) between an observed image point and the corresponding 3D point reprojected onto the camera frame.
    2. The reprojection error gives us a quantitative measure of the accuracy of the calibration (ideally it should be zero).
    3. $\pi(P_W^i, K, R, T)$ Reprojected point, $||p^i - \pi(P_W^i, K, R, T)||^2$
       ```

4. The reprojection error can be used to assess the quality of the camera calibration
5. what are the sources of the reprojection error? -> limitation of DLT
    1. Lens distortion
    2. Errors in corner detections
    3. Least square optimization is prone to noise

20. Non-linear calibration refinement
    1. The calibration parameters K, R, T determined by the DLT can be refined by minimizing the following cost: $K, R, T, \text{lens distortion} = argmin_{K,R,T,lens} \sum_{i=1}^{n} ||p^i - \pi(P_W^i, K, R, T)||^2$
    2. This time we also include the lens distortion (can be set to 0 for initialization).
    3. Can be minimized using **Levenberg-Marquardt** (more robust than Gauss-Newton to local minima).
21. Summary:
    1. M has 5 intrinsic parameters, 6 extrinsic parameter, $5 + 6 = 11$. M is a 3×4 matrix.
    2. Need at least 6 points to be able to estimate all the 12 unknowns.
    3. Note: each pair of point correspondences give rise to **two equations** since $\vec{x} = M\vec{X}$, is a homogenous equation, with x of size 3×1, M of size 3×4 and $\vec{X}$ of size 4×1, $x_1 = \frac{P_1^T \vec{x}}{P_3^T \vec{x}}$ and $x_2 = \frac{P_2^T \vec{x}}{P_3^T \vec{x}}$. Therefore, each point correspondences gives two constraints and we need at least 12 constraints to solve for the 12 unknowns, which means at least 6 points with known positions in 3D.
    4. $Mp = 0$, can be solved using SVD.

7. Zhang's method
    1. $QH = 0$
    2. H is homography transformation matrix (in 2D).
    3. Minimal solution
        1. $Q_{2n \times 9}$ should have rank 8 to have a unique (up to scale) non-trivial solution H.
        2. Each point correspondence provides 2 independent equations.
        3. Thus, a minimum of **4 non-collinear points** is required.
    4. Over-determined solution (in practice we always use this)
        1. $n \geq 4$ points.
        2. It can be solved through SVD (same considerations as Tsai's).
    5. H can be decomposed by recalling that: $\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} = \begin{bmatrix} \alpha_u & 0 & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & t_1 \\ r_{21} & r_{22} & t_2 \\ r_{31} & r_{32} & t_3 \end{bmatrix}$
    6. Differently form Tsai's method, the decomposition of H into K, R, T requires **at least two view**.
    7. In practice the more views the better, e.g. 20-50 views spanning the entire field of view of the camera for the best calibration results.
    8. Notice that now each view j has a different homography H^j (and so a different R^j and T^j). However, K **is the same for all views**.
    9. $\begin{bmatrix} h_{11}^j & h_{12}^j & h_{13}^j \\ h_{21}^j & h_{22}^j & h_{23}^j \\ h_{31}^j & h_{32}^j & h_{33}^j \end{bmatrix} = \begin{bmatrix} \alpha_u & 0 & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11}^j & r_{12}^j & t_1^j \\ r_{21}^j & r_{22}^j & t_2^j \\ r_{31}^j & r_{32}^j & t_3^j \end{bmatrix}$
    10. For each viewpoint we have constant intrinsic but different extrinsic.
    11. How to recover K, R, T from H and from multiple views?
        1. Estimate the homography H_i for each i-th view using the DLT algorithm.
        2. Determine the intrinsics K of the camera from a set of homographies.
            1. Each homography $H_i \sim K(\mathbf{r}_1, \mathbf{r}_2, \mathbf{t})$ provides two *linear* equations in the 6 entries of the matrix $B = K^{-T}K^{-1}$. Letting $\mathbf{w}_1 = K\mathbf{r}_1$, $\mathbf{w}_2 = K\mathbf{r}_2$, the rotation constraints $\mathbf{r}_1^T\mathbf{r}_1 = \mathbf{r}_2^T\mathbf{r}_2 = 1$ and $\mathbf{r}_1^T\mathbf{r}_2 = 0$ become $\mathbf{w}_1^T B\mathbf{w}_1 - \mathbf{w}_2^T B\mathbf{w}_2 = 0$ and $\mathbf{w}_1^T B\mathbf{w}_2 = 0$.
            2. Stack 2N equations from N views, to yield a linear system $A\mathbf{b} = \mathbf{0}$. Solve for $\mathbf{b}$ (i.e., B) using the SVD.
            3. Use Cholesky decomposition to obtain K from B.
        3. The extrinsic parameters for each view can be computed using K: $\mathbf{r}_1 \sim \lambda K^{-1} H_{i(:,1)}, \mathbf{r}_2 \sim \lambda K^{-1} H_{i(:,2)}$, $\mathbf{r_3} = \mathbf{r_1} \times \mathbf{r_2}$ and $T_i = \lambda K^{-1} H_i(:, 3)$, with $\lambda = 1/K^{-1}H_i(:, 1)$. Finally, build $R_i = (\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3)$ and enforce rotation matrix constraints.

8. Camera calibration vs Triangulation
    1. Both are for solving linear equations
    2. Camera calibration: Getting camera parameters from

    1. estimated 2D matching
    2. 3D measurements
3. Triangulation: Getting 3D points from
    1. estimated 2D matching
    2. camera parameters

9. Camera localization (or Perspective from n Points: PnP)
    1. from [1]:
        1. This is the problem of determining the **6 dof pose of a camera** (position and orientation) with respect to the world frame **from a set of 3D-2D point correspondences**.
        2. It assumes the **camera** to be **already calibrated**.
        3. In other words, the goal is **getting extrinsics (R and T) while intrinsics are given**.
        4. The **DLT can be used** to solve this problem **but is suboptimal**. We want to study **algebraic solutions** to the problem.
        5. How many points are enough? Since the H matrix has 8 dof, each 2D-2D point correspondences gives 2 equations, therefore we need at least 4 point correspondences (if coplanar) to get a unique solution.
        6. 1 point: infinite solutions.
        7. 2 Points: infinite solutions, but bounded.
        8. 3 Points (P3P problem), non collinear, up to 4 solution, L_A, L_B, and L_C are unknown variables.
        9. Algebraic approach: reduce to $4th$ order equation. From Carnot's theorem:
            1. $|AB|^2 = s_1^2 = L_B^2 + L_A^2 - 2L_BL_A\cos\theta_{AB}$
            2. $|AC|^2 = s_2^2 = L_A^2 + L_C^2 - 2L_AL_C\cos\theta_{AC}$
            3. $|BC|^2 = s_3^2 = L_B^2 + L_C^2 - 2L_BL_C\cos\theta_{BC}$
        10. It is known that n independent polynomial equations in n unknowns, can have no more solutions that the product of their respective degrees. Thus, the system can have a maximum of $2 \times 2 \times 2 = 8$ solutions. However, because every term in the system is either a constant or of second degree, for every real positive solution there is a negative solution.
        11. Thus, with 3 points, there are at most $(2 \times 2 \times 2)/2 = 4$ valid (positive) solutions.
        12. All PnP problems (solved by DLT, EPnP, or P3P algorithms) are prone to errors if there are outliers in the set of 3D-2D point correspondences.
        13. The RANSAC algorithm can be used, in conjunction with the PnP algorithm, to remove outliers.
        14. PnP with RANSAC can be found in OpenCV's (solvePnPRansac).
        15. Example of applications: SLAM in autonomous driving and vacuum robot.
        16. Summary: If a camera is calibrated, both EPnP and DLT can be used, only R and T need to be determined. In this case, when intrinsics is given, should we use DLT or EPnP?
        17. EPnP is up to $10\times$ more accurate and more efficient than DLT.
        18. EPnP: minimum number of points: 3 (P3P) + 1 for disambiguation.
        19. DLT: minimum number of points: 4 if coplanar (Zhang's method), 6 if non-coplanar (Tsai's method).
        20. The output of both DLT and EPnP can be refined via non-linear optimization by minimizing the sum of squared reprojection errors.
10. Real world camera calibration:
    1. We need to synchronize cameras in addition to calibrating them!
    2. Tricky way: clap (!) (casual setup).
    3. Advanced way: use synchronized triggers (trigger signal system).

Calibrated camera (i,e, intrinsic parameters are known)	Uncalibrated camera (i.e. intrinsic parameters unknown)
Either DLT or EPnP can be used	Only DLT can be used

11. Camera calibration (Zhang's and Tsai's) vs Epipolar geometry:
    1. Camera calibration needs matching points + 3D measurement (checkerboard)
    2. Epipolar geometry only need matching points (don't need 3D measurement)
12. from [2]:
    1. PnP is used to get the transformation from world to camera given the point correspondences of 3D-2D. Or more formally:

2. Camera pose estimation: Given a set of 3D points wrt. a world coordinate system and their projecting points in the image plane, compute the rigid transformation between the world and the camera coordinate systems.
3. For perspective imaging device, this is the so-called *perspective n point (PnP)* problem.
4. PnP problem: n 3D-2D correspondences (i.e. correspondences between 3D model points and image points) $\rightarrow$ Find Rotation and Translation (R, T).
5. EPnP: An accurate $O(n)$ solution to the PnP problem.
6. Note: The extrinsic camera parameter aka. camera pose is not constant unless the camera is placed fixed to a world coordinate frame.

13. Head pose estimation example

```
# -*- coding: utf-8 -*-
"""
Created on Thu Mar  5 14:15:24 2026

@author: reina
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt
from math import cos, sin, atan2, sqrt


def rad2deg(x):
    return x / np.pi * 180


def homogenize(x):
    # x is of size (d, n)
    return np.vstack((x, np.ones((1, x.shape[1]))))


def dehomogenize(x):
    # x is of size (d+1, n)
    return x[:-1, :] / x[-1, :]


def pose_estimation(img, face2d, face3d, color):
    _, rotvec, transvec = cv2.solvePnP(face3d.T, face2d.T, K, None)
    R, _ = cv2.Rodrigues(rotvec)
    print(R)
    r11 = R[0, 0]; r21 = R[1, 0]
    l = np.sqrt(r11**2 + r21**2)
    r32 = R[2, 1]; r33 = R[2, 2]; r31 = R[2, 0]
    r23 = R[1, 2]; r22 = R[1, 1]
    if l >= 1e-6:
        pitch = atan2(r32, r33)
        yaw = atan2(-r31, l)
        roll = atan2(r21, r11)
    else:
        pitch = -atan2(r23, r22)
        yaw = atan2(-r31, l)
        roll = 0
    yaw = rad2deg(yaw)
    pitch = rad2deg(pitch)
    roll = rad2deg(roll)
    print(yaw, pitch , roll)
    # inrinsic matrix
```

```
    K_i = K
    # extrinsic matrix
    K_o = np.zeros((4, 4))
    K_o[:3, :3] = R
    K_o[:3, 3] = transvec.squeeze()
    K_o[3, 3] = 1.
    print(K_o)
    pi_0 = np.zeros((3, 4))  # Canonical projection matrix i.e. 3D-2D
    pi_0[:3, :3] = np.eye(3)
    face3d_hom = homogenize(face3d)  # (4, n)
    face2d_repr = dehomogenize(K_i @ pi_0 @ K_o @ face3d_hom)  # (3, 3) (3, 4), (4, 4) (4, n)
    print(face2d_repr.shape)

    for i in range(face2d_repr.shape[1]):
        cv2.circle(img_copy, (int(face2d_repr[0, i]), int(face2d_repr[1, i])), 1, color, 2)
    cv2.imshow('emma_watson', img_copy)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    ### Calculate reprojection error ###
    # repr_error = np.sum((face2d - face2d_repr)**2) / face2d.shape[1]
    repr_error = np.sum(np.mean((face2d - face2d_repr)**2, axis = 1))
    print(face2d_repr.shape)
    print(repr_error)


if __name__ == '__main__':
    """
    Pose estimation of world points where
    world_points = face_points.
    Given intrinsic camera matrix, 3D world points,
    and the corresponding 2D camera points.
    """

    img_name = 'emma_watson'
    face2d = np.load(img_name + '.npy')
    face2d = face2d.astype('float32')
    face3d = np.load('basel_68_pts.npy')
    print(face2d.shape)  # (d, n)
    print(face3d.shape)  # (d, n)
    # (d, n) points format commonly used in 3D vision
    # Do not change the following two lines
    face3d[1,:] *= -1
    face3d[2,:] *= -1
    print('face3d.shape', face3d.shape)

    # TODO1: Display the 2D landmarks
    # enter your code here
    img = cv2.imread('emma_watson.jpg')
    # img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # Only used when displaying using matplotlib imshow
    img_copy = img.copy()
    for i in range(face2d.shape[1]):
        cv2.circle(img_copy, (int(face2d[0, i]), int(face2d[1, i])), 1, (0, 255, 0), 2)
    cv2.imshow('emma_watson', img_copy)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    h, w = img.shape[:2]
    # Construct pseudo intrinsic matrix
```

```
f = np.maximum(h, w)
cx = w / 2
cy = h / 2
K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
print(K)

### Pose Estimation using 68 points ###
# repr_error : 104.44326337181667
pose_estimation(img_copy, face2d, face3d, (255, 0, 0))
### Pose Estimation using 51 points ###
# repr_error : 122.04128202553773
pose_estimation(img_copy, face2d[:, :51], face3d[:, :51], (0, 0, 255))
```

References:

1. Introduction to Computer Vision (2025-2), Prof Gyeonsik Moon, Korea University.
2. CSIE 5429 3D-DLCV, National Taiwan University, Spring 2021
3. Barycentric coordinate system https://en.wikipedia.org/wiki/Barycentric_coordinate_system
4. Vincent Lepetit · Francesc Moreno-Noguer · Pascal Fua, "EPnP: An Accurate O(n) Solution to the PnP Problem", International Journal of Computer Vision ,February 2009